

TUGAS KECIL II — IF2211 STRATEGI ALGORITMA

3D Voxelization using Octree

Semester II 2025/2026

Made Branenda Jordhy — 13524026
Valentino Daniel Kusumo — 13524104

School of Electrical Engineering and Informatics
Institut Teknologi Bandung

March 25, 2026

Abstract

Contents

1	Introduction	3
2	Theoretical Background	4
2.1	Voxelization	4
2.2	Octree	4
2.3	Divide and Conquer	4
2.4	Separating Axis Theorem (SAT)	4
3	Divide and Conquer Algorithm	4
3.1	High-Level Steps	4
3.2	Pseudocode	4
3.3	Complexity Analysis	4
4	Implementation	4
4.1	System Architecture	4
4.2	Program Flow	4
4.3	Key Implementation Details	4
5	Results and Analysis	4
5.1	Test Case 1	4
5.2	Test Case 2	4
5.3	Test Case 3	4
5.4	Test Case 4	4
5.5	Test Case 5	4
5.6	Performance Summary	4
6	Bonus Features	4
6.1	Concurrency	4
6.2	Interactive 3D Viewer	4
7	Conclusion	4
	Repository	5

1 Introduction

Dalam *computer graphic*, objek 3D itu umumnya disimpan sebagai *polygon mesh*. Format ini fleksibel dan ringan tetapi tidak selalu cocok untuk kebutuhan tertentu. Seperti yang kita tau, *grid based physics simulations*, *3D printing*, hingga game seperti Minecraft, itu semuanya membutuhkan representasi volumetrik yang lebih terstruktur. Di sinilah proses *voxelization* digunakan yaitu mengubah *polygon mesh* menjadi kubus kecil seragam yang disebut *voxel*.

Problem utamanya adalah kita diberikan sebuah file `.obj` berisi *polygon mesh* dan program harus menghasilkan `.obj` baru yang mana seluruhnya tersusun dari voxel yang seragam yang merpresentasikan objek aslinya. Namun, ada obstacle yang terletak pada scale, pada depth octree yang tinggi, jumlah candidate voxel cenderung tumbuh secara eksponensial (8^N pada depth N).

Lalu bagaimana caranya kita develop? Ya, dengan menggunakan paradigma *Divide and Conquer* melalui *Octree* (ya sesuai spesifikasi tucil...). Ruang 3D dibungkus oleh bounding box (AABB) berbentuk cubic lalu dipotong secara rekursif menjadi 8 sub-cubic pada setiap level. Di setiap step, kita melakukan uji perpotongan antara segitiga mesh dan kotak menggunakan *Separating Axis Theorem* (SAT). Kotak yang tidak bersinggungan dengan segitiga manapun langsung di-*pruned* sehingga hanya bagian yang relevan yang ditelusuri lebih dalam. Proses ini naturally membentuk paradigma *Divide and Conquer*, divide ruang, conquer setiap bagian independently, lalu terakhir tinggal combine.

Program yang kita develop di sini sepenuhnya dalam `C++`. Selain fungsionalitas utama (wajib), kami juga mengimplementasikan dua fitur bonus yaitu *concurrency* untuk paralelisasi rekursif pada octree menggunakan `std::thread` serta *interactive 3D viewer* dengan raylib yang memungkinkan user melakukan eksplorasi hasil voxelization.

2 Theoretical Background

2.1 Voxelization

2.2 Octree

2.3 Divide and Conquer

2.4 Separating Axis Theorem (SAT)

3 Divide and Conquer Algorithm

3.1 High-Level Steps

3.2 Pseudocode

3.3 Complexity Analysis

4 Implementation

4.1 System Architecture

4.2 Program Flow

4.3 Key Implementation Details

5 Results and Analysis

5.1 Test Case 1

5.2 Test Case 2

5.3 Test Case 3

5.4 Test Case 4

5.5 Test Case 5

5.6 Performance Summary

6 Bonus Features

6.1 Concurrency

6.2 Interactive 3D Viewer

7 Conclusion

Repository

https://github.com/ethj0r/Tucil2_13524026_13524104